

Gra wylawianie skarbu

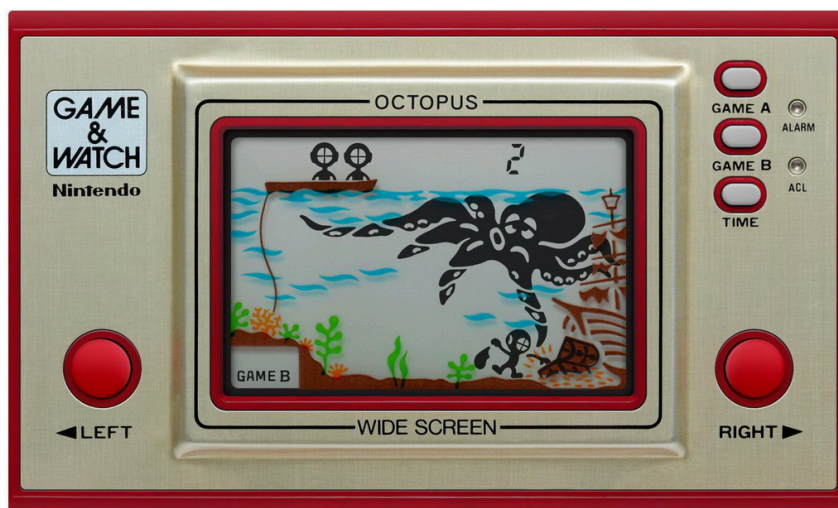
Projekt GUI

Octopus to jedna z klasycznych gier z serii Game&Watch, w której wciela się w nurka próbującego wydobyć skarby z dna oceanu i bezpiecznie dostarczyć je do łodzi, unikając macek gigantycznej ośmiornicy. Każde zanurzenie wiąże się z ryzykiem utraty życia, ponieważ ramiona ośmiornicy poruszają się według określonych wzorców i mogą zablokować drogę powrotną.

Celem gry jest zdobycie jak największej liczby punktów poprzez wielokrotne pobieranie skarbów z dna oceanu i dostarczanie ich na powierzchnię. Gra stopniowo zwiększa poziom trudności wraz ze wzrostem liczby punktów.

Sterowanie odbywało się za pomocą dwóch przycisków umożliwiających ruch w lewo i w prawo. Przykładową rozgrywkę można zobaczyć pod linkem:

<https://www.youtube.com/watch?v=U8Zj3YXQzVU>



Rysunek 1: Poglądowy wygląd gry

1 Opis zasad gry

- Gracz rozpoczyna rozgrywkę z trzema żywami.
- Cel gry: gracz steruje nurkiem poruszającym się pomiędzy łodzią a skarbem znajdującym się na dnie oceanu celem wydobywania całego skarbu spoczywającego w wraku statku.
- Nurek może poruszać się wyłącznie po zdefiniowanych pozycjach planszy.
- Sterowanie: Gra będzie obsługiwana za pomocą klawiatury, dwoma przyciskami – jeden do poruszania się w lewo, drugi do poruszania się w prawo. Gracz będzie musiał używać tych przycisków w odpowiednim momencie, aby uniknąć pochwycenia przez mackę ośmiornicy.

- Przeszkody: W trakcie nurkowania gracz musi unikać macek ośmiornicy. Macki poruszają się cyklicznie i mogą wysuwać się lub chować. Dotknięcie macki powoduje utratę jednego życia.
- Punkty: Gracz zdobywa punkty za dotarcie do wraku statku i dostarczenie worka ze skarbem na łódź.
- Trudność: Gra stopniowo zwiększa swoją trudność poprzez zwiększenie szybkości ruchu macek.
- Koniec gry: Gra kończy się w momencie, gdy macka trzykrotnie dotknie nurka lub gdy gracz osiągnie 999 punktów.

2 Opis projektu

Wykorzystując komponenty biblioteki do budowy graficznych interfejsów użytkownika **SWING** zaimplementuj autorską wersję gry Octopus, przyjmując następujące założenia:

- logikę gry i reprezentację planszy gry należy umieścić w pakiecie **projekt2526.game** jako klasę **Board** implementującą jednocześnie interfejs **KeyListener**. Wciskanie klawiszy sterujących w aplikacji będzie obsługiwane przez bezpośrednią implementację metod;
- plansza gry będzie reprezentowana jako dwuwymiarowa tablica szarpana zmiennych **int** o rozmiarze 6×2, 3, 4, 5, 4, 3 elementów, reprezentującą stan oraz zmienną gdzie element tablicy o indeksach 0×0 reprezentuje łódź nurków, element tablicy o indeksach 0×1 reprezentuje pozycję zanurzonego nurka w wodzie, a pozostałe pola opisują stan macek;
- pozycje macek ośmiornicy powinny zmieniać się automatycznie w określonych odstępach czasu;
- każdy segment macki może znajdować się w jednym z dwóch stanów: schowana lub wysunięta;
- po osiągnięciu określonych progów punktowych gra powinna zwiększać poziom trudności poprzez: zwiększenie szybkości animacji, częstsze wysuwanie macek;
- jeżeli nurek dotrze do wraku i pobierze skarb, generowane jest zdarzenie **PlusOneEvent**;
- dostarczenie worka do łodzi powoduje trzykrotne wywołanie zdarzenia **PlusOneEvent**;
- w przypadku detekcji zetknięcia postaci gracza z macką ośmiornicy nastąpi utrata jednego punktu życia, natomiast jeśli był to ostatni punkt, wygenerowane zostanie zdarzenie **ResetEvent**;
- część graficzna gry będzie umieszczona w pakiecie **projekt2526.pres**
- na część graficzną składają się:
 - komponent graficzny **SpriteRenderer** odpowiadający za reprezentację planszy gry (czarne elementy ekranu, patrz rysunek 1);
 - komponent graficzny **JPanel** odpowiadający za kolorowe elementy ekranu;

- komponent **JTable** odpowiadający za przedstawienie pojedynczej cyfry reprezentowanej jako siedmio-segmentowy element wykorzystujący MVC do przedstawienia pojedynczej cyfry;
- licznik punktów (centrum ekranu), zbudowany z 3 komponentów **JTable** (reprezentujących odpowiednio od prawej do lewej rzędy jedności, dziesiątek i setek), z których każdy będzie realizowany przez własną implementację klasy o nazwie **SevenSegmentDigit**. Obiekty tej klasy będą ze sobą sekwencyjnie powiązane, tak aby każdy następny nasłuchiwał na zdarzenia poprzedniego. Np. jeżeli obiekt klasy **SevenSegmentDigit** reprezentujący rząd jedności wygeneruje zdarzenie **PlusOneEvent** to obiekt reprezentujący rząd dziesiątek zwiększy swój stan o jeden, itd adekwatnie do kontekstu przychodzących zdarzeń.

Obiekt będzie odbierał i generował zdarzenia zgodnie z delegacyjnym modelem zdarzeń. Koniecznym zatem będzie implementacja mechanizmu generującego i przyjmującego następujące zdarzenia:

- **StartEvent** – ustalający stan cyfry na 0 i generując zdarzenie **StartEvent** rozsyłane do wszystkich nasłuchujących komponentów;
- **PlusOneEvent** – dodający kolejny punkt i zwiększający cyfrę o jeden. Jeżeli zwiększana cyfra będzie cyfrą 9 to powraca do reprezentowania cyfry 0, jednocześnie generując zdarzenie **PlusOneEvent** rozsyłane do wszystkich nasłuchujących komponentów;
- **ResetEvent** – wyłączający wszystkie segmenty cyfry, jednocześnie generując zdarzenie **ResetEvent** rozsyłane do wszystkich nasłuchujących komponentów;
- obsługa klawiatury będzie realizowana za pomocą implementacji interfejsu **KeyListener** i zakłada wykorzystanie klawiszy:
 - **s** – rozpoczęcie rozgrywki;
 - **a** – przesunięcie postaci w lewo;
 - **d** – przesunięcie postaci w prawo.

Otrzymanie określonego zdarzenia będzie musiało prowadzić do odświeżenia oblicza tego komponentu. Przykład segmentowej cyfry przedstawiono na rysunku 2



Rysunek 2: Przykład siedmiosegmentowych cyfr

- uruchomienie rozgrywki następuje po wciśnięciu i puszczeniu klawisza **s**, skutkiem czego będzie uruchomienie lub wznowienie niezależnego od interfejsu graficznego

wątku (realizowanego przez własną implementację klasy **Thread**). Celem tego wątku będzie:

- ▶ cykliczne generowanie zdarzenia **TickEvent** i rozsyłanie go do wszystkich nasłuchujących elementów gry, zgodnie z delegacyjnym modelem zdarzeń;
- ▶ interwał pomiędzy cyklami wątku będzie się stopniowo zmniejszał;
- ▶ wątek zostanie wstrzymany, gdy:
 - obiekt klasy **SevenSegmentDigit** reprezentujący ostatnią (rząd setek) cyfrę wygeneruje zdarzenie **PlusOneEvent**;
 - gracz straci ostatni punkt życia i obiekt klasy wątku otrzyma zdarzenie **ResetEvent**;

Niezależnie od przyczyny wstrzymania wątku interwał pomiędzy cyklami wątku powróci do początkowej wartości.

Wykorzystaj wzorzec programistyczny **Singleton**, aby ponowne lub wielokrotne kliknięcie przycisku **s** nie skutkowało uruchomieniem kolejnych wątków.

- w toku rozgrywki występują następujące zdarzenia:
 - ▶ **TickEvent**, który powiadamia klasę reprezentującą planszę iż należy przesunąć postać gracza i macki;
 - ▶ wciskanie klawiszy **a** i **d** będzie obsługiwane przez bezpośrednią implementację interfejsu **KeyListener** w klasie **Board**.

Sugestie

1. zaleca się znaleźć w internecie obrazek przedstawiający wszystkie wyświetlane na czarno elementy gry i „pociąć” je;
2. przedstawianie cyfr seven segment digit jako elementu **JTable** wymaga przemyślenia jakie stany i części cyfry będą wyświetlane w jakich komórkach.

Wymagania formalne

Zgodnie z zasadami zaliczenia przedmiotu ocenie będą podlegać prace studentów, którzy uzyskali w trakcie semestru co najmniej 80% punktów.

Korzystanie ze wsparcia AI w projekcie jest dopuszczalne pod warunkiem pełnego zrozumienia generowanego kodu. Student musi być w stanie samodzielnie wyjaśnić działanie każdej klasy, metody oraz zastosowanej koncepcji programistycznej.

Nieumiejętność wytłumaczenia (lub błędne wytłumaczenie) działania jest tożsame z porażką obrony projektu.

Powstały jako projekt kod źródłowy oraz pliki graficzne muszą być dostarczone w postaci pojedynczego pliku archiwum **.zip** o nazwie **sXXXXX_projekt2526.zip**, gdzie **XXXXX** jest numerem studenta.

Rozwiązanie w postaci archiwum należy przesłać poprzez udostępniony assignment do dnia **19.06.2026 r. (Piątek) godz 23:59** na platformie Teams.

W projekcie wymagane jest wykorzystanie wszystkich zagadnień określonych w treści zadania. Należy zagwarantować, że cały kod będzie opierał się tylko na standardowej bibliotece Java. Zawarcie elementów spoza zakresu przedmiotu wymaga uzyskania zgody prowadzącego.

Uwaga: niespełnienie wymagań formalnych będzie skutkowało wykluczeniem oceny 5.

Projekt jest uznawany za przesłany do oceny dopiero po dołączeniu pliku oraz kliknięciu przycisku „Turn in”.

Projekt musi zostać przesłany przed upływem terminu podanego w zadaniu.

Każda próba oddania projektu po upływie terminu zostanie automatycznie odrzucona przez platformę MS Teams.

Rozwiązywanie problemów technicznych

Jeśli student napotka problemy techniczne uniemożliwiające prawidłowe przesłanie projektu, należy nagrać krótki film (30–40 sekund) obejmujący cały ekran (pulpit systemu). Film powinien wyraźnie pokazywać:

- próbę przesłania projektu
- komunikat o błędzie wygenerowany przez system po kliknięciu przycisku „Turn in”

Nagranie to wraz z archiwum zip musi zostać przesłane prowadzącemu przed upływem terminu oddania projektu poprzez:

- wiadomość na kanale grupowym MS Teams lub
- wiadomość e-mail na wyznaczony adres kontaktowy

Znacznik czasu wiadomości na MS Teams lub e-maila musi być wcześniejszy niż termin oddania projektu.

Proszę uwzględnić ten wymóg przy planowaniu pracy nad projektem i zapewnić sobie wystarczającą ilość czasu, która pozwoli na właściwe udokumentowanie oraz terminowe zgłoszenie ewentualnych problemów technicznych, gdyby takowe wystąpiły.

Ocena projektu

Projekt będzie oceniony według poniższego zestawu kryteriów

Kategoria	Maksymalna liczba punktów
1. Logika gry – klasa Board	20
2. Mechanika macek i poziom trudności	10
3. System zdarzeń delegacyjnych	20
4. SevenSegmentDigit i licznik punktów	20
5. Komponenty graficzne	10
6. Wątek gry i wzorzec Singleton	10
7. Sterowanie klawiaturą	5
8. Wymagania formalne i jakość kodu	5
RAZEM	100 pkt.